Comparing Matrix Multiplication Implementations Across CPU and GPU Platforms

Bassil Obeidi

October 24, 2026

Abstract

The purpose of this project is to act as an exploratory analysis into the world of parallel computing. We all hear about libraries like PyTorch, Numpy, even CUDA, but usually we simply invoke the libraries without thinking about whats going on under the hood. I must note that this analysis as it stands is extremely limited, however it has acted as an invaluable introduction to GPU kernels. I also want to note that I did not rely on any AI-generated code for the kernel codes, as I wanted to subject myself to the pedagogical (and frustrating) experience of writing these kernels the old fashion way, using documentation and video guides. I did however use AI-assisted code for writing the benchmarking script, generating the visuals and formatting this LaTeX report. This project compares various matrix multiplcation implementations across various GPU and CPU platforms. Using a RTX 4070 Super and a Intel i5 13600k, I compare computational speed for MatMul across CUDA, Triton, Pytorch, Numpy and Python implementations. Overall, a blocked Triton kernel outperformed all other implementations, yet suffered a massive warmup penalty despite this performance. PyTorch was the close runner up, with CUDA in third. These experiments used a fixed 1024×1024 matrix, and fixed block/grid sizes of 32×32 for Triton and CUDA. Overall, this project is an initial study with near limitless potential for expansion and additional experiments.

1 Introduction

Matrix multiplcation is the mathematical operation underpinning all modern AI and ML. Despite this, very few people legitimately understand what is happening under the hood, to turn an $O(n^3)$ complexity algorithm into one of the most efficient, optimized and most frequently used operations in computing. Strassen's algorithm reduced the exponent to $\log_2 7 \approx 2.8074$, and recent OpenAI work established an upper bound of $9/4 = 2.25$ for the matrix-multiplication exponent over the complex numbers [4, 5]. Hopefully, after reading this "study" a reader might gain some insight into the poorly documented and rarely thought about world of parallel computing.

2 Implementations

This section will cover the 6 different MatMul implementations, as well as cover the differing kernel designs for Triton and CUDA.

2.1 Python

This is the true O(n^3) monstrosity that we all know and love. It ain't efficient, but it works. Or that at least what I imagine someone who uses Python For loops and arrays to do MatMul on a

1024×1024 matrix would say. To make matters worse, Python by default is single-threaded and attempts to make the MatMul multi-threaded using Python's `Threading` module will be ineffective due to the Global Interpreter Lock. Anyways, optimizing MatMul using Python is it's own rabbit hole, worth a seperate research project (something something multiprocessing module and chunking). Let's just summarize by saying if you're doing MatMul using Python loops for anything other than a LeetCode problem, something's gone horribly wrong.

2.2 Numpy

Not much to say here. It works, it's fast and it doesn't require downlading a massive CUDA library. I'm not entirely sure how they managed it (future work), but props!

2.3 PyTorch

Ah PyTorch. I usually just import you and forget about it. From what I knew before this project, PyTorch has a CPU version as well as a GPU version. The GPU version requires a CUDA installation, the CPU version doesn't. After a bit of external research, it seems that PyTorch achieves faster than NumPy computation on "large" matrices, by using OpenMP and Intel MKL (Math Kernel Library) built in libraries. These libraries enable highly efficient multi-threaded operations on CPUs.

For the PyTorch GPU version, we all know that PyTorch GPU requires a CUDA install, and therefore logically, it generates some type of CUDA kernel to compute the MatMul. After some external research, it turns out that PyTorch doesn't actually generate and CUDA kernels itself, instead it calls CuBLAS (NVIDIA CUDA Basic Linear Algebra Subprograms). CuBLAS essentially contains a whole list of prewritten CUDA kernels, which it chooses the "optimal" one for your purposes by running some type of runtime heuristics, hadware runtime benchmarks and workspace contrainst. Neat!

2.4 Triton

Where the fun begins. Or pain. I'm not sure. Triton is an open-source GPU-programming library created by OpenAI back in 2021. From what I understand, its purpose is to enable developers to write custom GPU kernels (in Python syntax), without any C++ or CUDA knowledge prerequisites. Importantly, Triton kernels rival or match native CUDA C++ kernels. I chose to write two different Triton kernels. Compare to CUDA, Triton abstracts a decent amount of concepts such as memory management, individual thread handling and hardware commands. The fundemental procesing unit in Triton is a PID (Program ID), which I've learned is **not** equivalent to a thread. Triton also handles memory addressing using something called strides, which are basically a unit that describes how many memory indices are required to reach the next row-wise or column-wise element.

2.4.1 Naive Triton Kernel

This kernel assigns a unique PID to each element of the output array. So for a 1024×1024 output matrix, over a million PIDs get spawned in. Turns out this isn't that efficient (hey its called Naive for a reason). Importantly PIDs are **not the same as CUDA THREADS**.

2.4.2 Blocked Triton Kernel

This kernel splits the output array in "blocks" to which each block is assigned one PID. This drastically reduces the number of PIDs spawned in. Each PID is reponsible for "striding" across

the input arrays and accumulating the dot product values for each output element in its block. This approach results in much more efficient memory (re)usage through something called coalesced memory access, which is essentially less GPU DRAM trips (which are really slow). Additionally, neighbouring output elements share the same input rows/columns which means less local SRAM trips. Different block sizes and shapes result in different performance levels, but as a rule of thumb, square blocks are best.

2.5 CUDA

This kernel assigns a single GPU thread to each element of the output array. However, GPU threads are **not** Triton PIDs, so this doesn't end up being that bad. However, the memory efficiency of this approach is not great. Each thread is making individual requests that often wipe out past cached elements, which could be used by subsequent threads, resulting in unecessary trips to DRAM (SLOW).

2.5.1 Naive CUDA

This kernel sections off the output array into a grid, and each grid is assigned one thread. Very similar in concept to the blocked Triton kernel. Localized "tiles" cooperatively load groups of input elements to increase gpu cache usage and decrease global memory traffic

3 Experimental Method

3.1 Hardware and software

The project ran on Windows 11 Home with an NVIDIA GeForce RTX 4070 SUPER (12 GB GDDR6X) and an Intel Core i5-13600K (14 cores: 6 performance cores and 8 efficiency cores; 20 threads).

The software environment used Python 3.12, CUDA Toolkit 12.6, and Visual Studio 2022. The Python packages were PyTorch 2.14.1 with CUDA 12.6, Triton for Windows 3.8.0.post29, NumPy 2.5.4, and Ninja 1.13.2.

3.2 Workload and correctness

Every implementation multiplied two fixed 1024×1024 matrices whose entries were all 2.0. NumPy, PyTorch, Triton, and CUDA used 32-bit floating-point inputs and outputs; the pure-Python implementation used nested lists of Python floating-point values. The saved run contains 30 timed samples for each implementation except pure Python, which has one.

The benchmark code does not compare the computed outputs with a reference result, so correctness was not systematically validated and no numerical tolerance was applied.

3.3 Timing procedure

CPU operations were timed with Python's `time.perf_counter()`. GPU samples were measured with `triton.testing.do_bench_cudagraph()`, which times CUDA graph replays and reports an average operation time. GPU inputs were created and transferred before the timed samples. The Triton and CUDA kernels reuse preallocated output matrices; PyTorch creates its output as part of each matrix multiplication call.

Each implementation also had a separate warm-up call. GPU warm-ups were synchronized before and after timing. The CUDA extension was loaded before its warm-up, while the first Triton call could include compilation. Warm-up times therefore include different work across implementations.

3.4 Precision

The array and tensor implementations used 32-bit floating-point inputs. The CUDA C++ kernels multiply and accumulate in FP32. PyTorch's matrix multiplication precision was not explicitly configured, so it used the default `highest` setting, with TF32 disabled [2]. The blocked Triton kernel calls `tl.dot` without setting an input precision; on this NVIDIA GPU, Triton's default is TF32, with an FP32 accumulator [1]. The naive Triton kernel accumulates its scalar products in FP32. PyTorch uses full FP32 precision.

4 Results

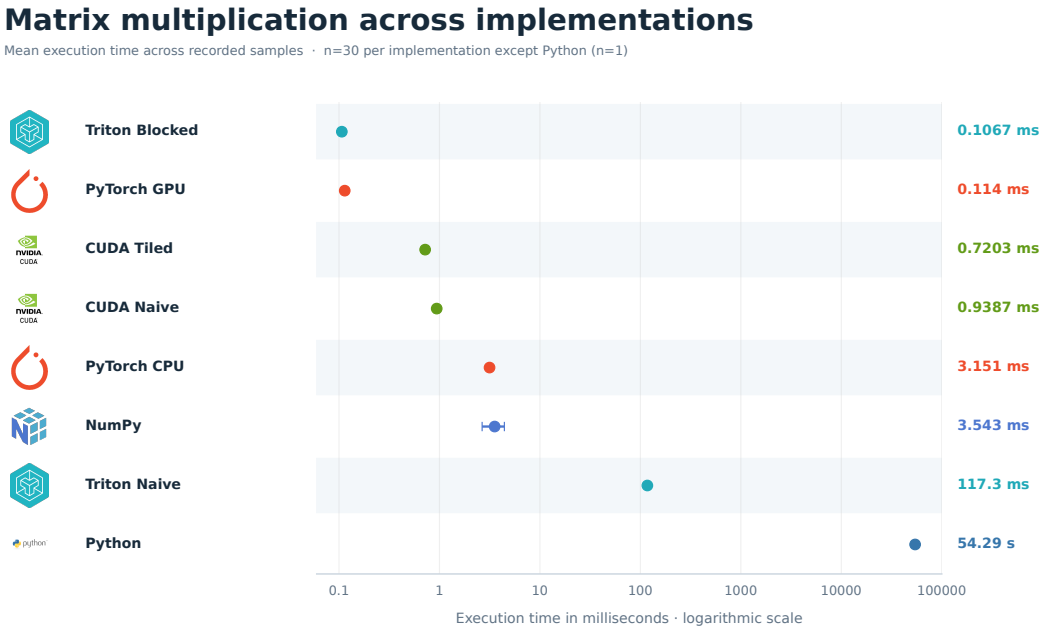
The blocked Triton kernel had the lowest mean time in the saved run at 0.1067 ms, with PyTorch GPU close behind at 0.1140 ms. The difference between their means was small compared with the variation across their samples, so I would describe these results as nearly tied. The naive Triton kernel took 117.34 ms on average, while the blocked version took 0.1067 ms. The tiled CUDA kernel averaged 0.7203 ms, compared with 0.9387 ms for naive CUDA.

On the CPU, PyTorch averaged 3.151 ms and NumPy averaged 3.543 ms. The pure-Python implementation took 54.29 s in its single measured sample.

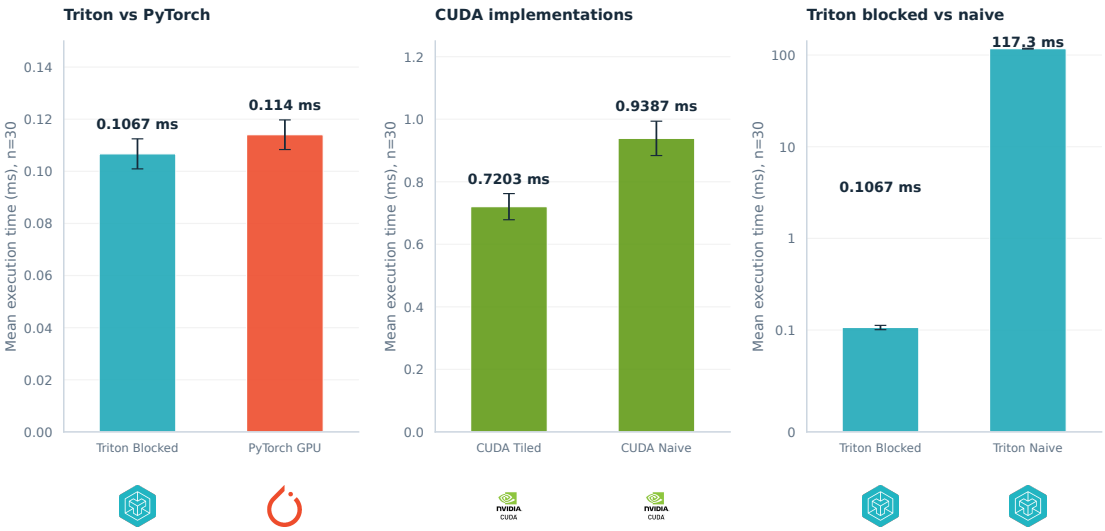
Table 1: Mean and median measured execution times. Python has one sample; all other implementations have 30.

Implementation	Mean (ms)	Median (ms)
Python	54,285.96	54,285.96
NumPy	3.543	3.275
PyTorch CPU	3.151	3.169
PyTorch GPU	0.1140	0.1118
Triton Blocked	0.1067	0.1045
Triton Naive	117.34	117.14
CUDA Naive	0.9387	0.9160
CUDA Tiled	0.7203	0.7024

The results show how the implementations performed for this fixed input and setup. They do not isolate the effect of kernel design from differences in library implementation and precision.



(a) Mean execution time across implementations. The horizontal axis is logarithmic. Python has one measured sample; other implementations have 30.



(b) GPU execution time comparisons. The Triton blocked-versus-naive panel uses a symmetric logarithmic vertical scale.

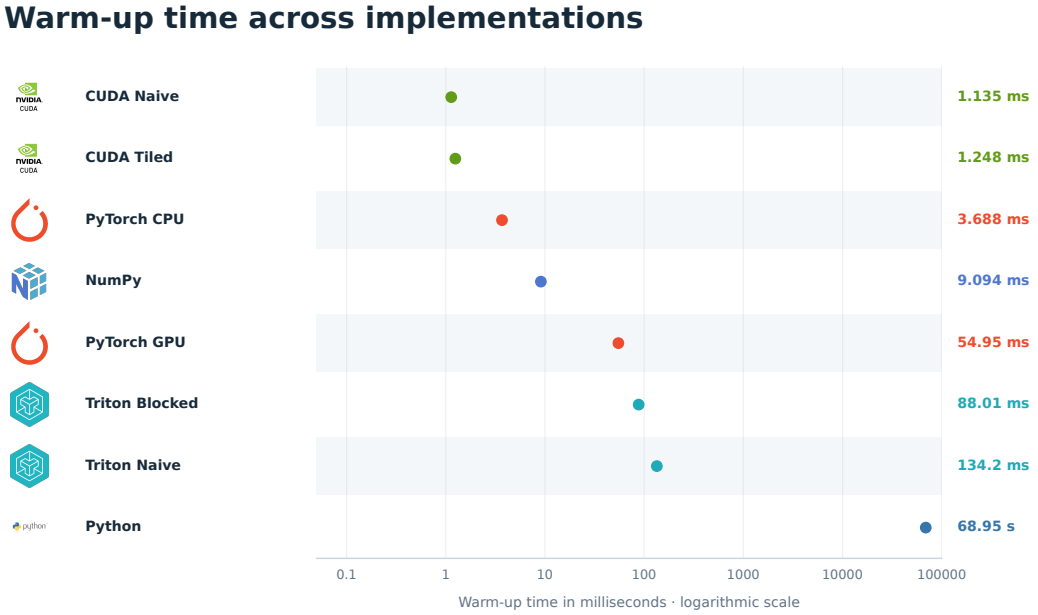
Figure 1: Overall and GPU-focused timing results.

The first two GPU pairs show that blocked Triton and PyTorch GPU were close: their mean times differed by about 0.007 ms, which is similar in size to the variation across their samples. CUDA tiled averaged 0.720 ms compared with 0.939 ms for CUDA naive. The largest gap was between the two Triton kernels: the blocked version averaged about 1,100 times faster than the naive version in this run.

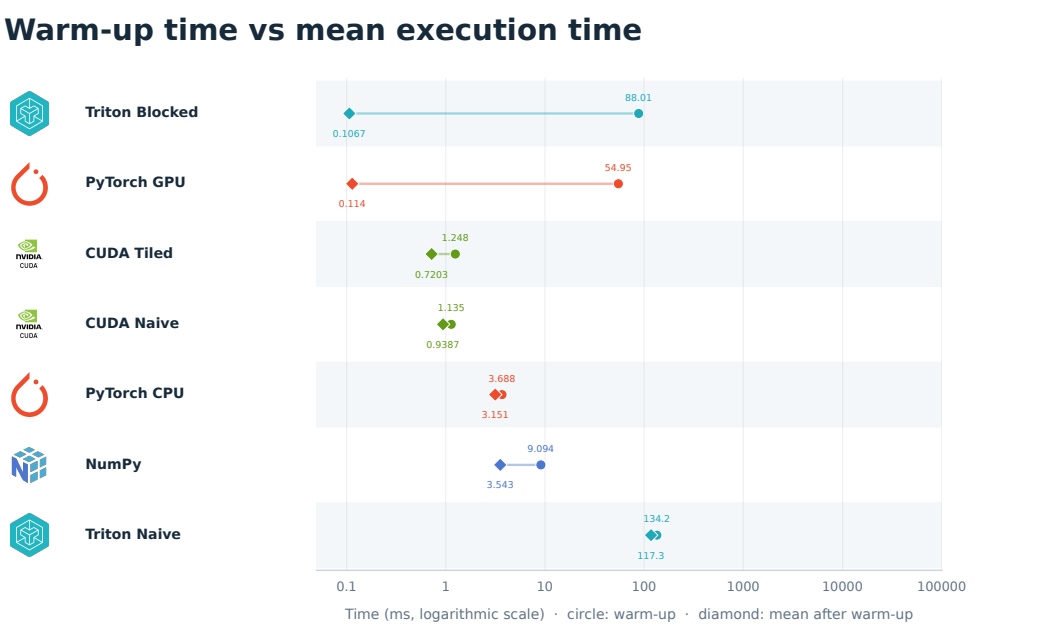


Figure 2: Individual measured timing samples and their distributions.

The distributions show that PyTorch CPU timings were tightly grouped, while NumPy varied more, with samples from 2.852 to 7.248 ms. This spread pulls the NumPy mean above its median. The PyTorch GPU and blocked Triton samples cluster closely, matching the small difference between their summary statistics.



(a) Separately recorded warm-up times.



(b) Warm-up time compared with mean measured time for implementations with repeated samples.

Figure 3: Warm-up measurements and their relationship to the timed samples.

Warm-up times varied substantially across implementations. The Triton naive and blocked kernels took about 134 ms and 88 ms, respectively, while the PyTorch GPU warm-up took about 55 ms; the pure-Python warm-up took nearly 69 seconds. Triton compilation can contribute to its first-call warm-up, while CUDA extension loading happened before the CUDA warm-up was timed. These values help explain first-use cost.

5 Discussion

There were three main findings that I'd like to explore.

5.1 Why is naive Triton so SLOW?

This was a surpising finding. The naive Triton kernel is around 1000x slower than the blocked version. Slower even than CPU implementations. Before completing this project I assumed that even the worst GPU kernel would be miles faster than a CPU implementation. Turns out that assumption was wrong. So what went wrong? After looking online there a number of explanations, and likely all of them contribute. Things like severy memory stall, low GPU tensor core usage, and globabl memory access overhead. Additionally, this kernel does element by element $*$ Python multiplication, which is apparently super unoptimized versus something like tl.dot.

5.2 Why is blocked Triton so FAST?

Another suprising find. From my preconceptions, I totally expected CUDA to beat Triton, or at least the tiled cuda to beat Triton. Furthermore, I also expected Pytorch to beat my blocked Triton kernel. As to why they didn't, im not entirely sure. However I'd like to dedicate future research to figuring out why.

5.3 Triton and PyTorch sure love to warmup! CUDA is ICE COLD

Triton completely dominated the warmup graph (in a bad way). Despite blocked and naive Triton varying 1000x, their warmup times were within the same order of magnitude. Similar behaviour was exhibited by PyTorch GPU which might related. CUDA suffers little to no warmup penalty, seemingly the opposite of Triton and PyTorch GPU.

Overall, all the findings above most definitely warrant further exploration into the causes and explanations for such interesting and varied behaviour.

6 Limitations

This was a small experiment on one computer, one GPU, and one matrix size. Both input matrices were filled with the same value, so the results do not show how these implementations behave on varied data, other dimensions, or other hardware. I also did not compare any output against a reference result, so the benchmark records speed without a systematic correctness check.

The timing methods were not identical across devices. CPU calls were measured with a host timer, while GPU samples used CUDA graph replays; these GPU measurements emphasize repeated execution and do not include the usual per-call Python launch overhead. The GPU implementations also used different output handling, and the separate warm-up calls included different work, such as Triton compilation. The reported times are therefore useful for comparing the recorded setup, but are not a fully controlled end-to-end comparison.

Precision was another difference. PyTorch used its default FP32 matmul setting, while the blocked Triton kernel used TF32 inputs with FP32 accumulation. In particular, the blocked-versus-naive Triton result combines a change in kernel design with a change in precision, so this run cannot attribute the full speed difference to blocking alone.

7 Future Work

The next step would be to test more input sizes and vary the block and grid sizes, both manually and with Triton and NVIDIA's tuning tools. I would also check each output against a reference, measure its numerical error, and repeat the benchmarks with different precision settings. I would study how the libraries choose and implement their matrix multiplication kernels in more depth, so explanations for the measured results are based on what the libraries are actually doing rather than speculation. Finally, I would compare these implementations with other GPU libraries such as CUTLASS and AMD ROCm.

8 Conclusion

GPUs are extremely fast processing units, however the way in which they are programatically utilized defines their real world performance. Different libraries offer different methods of defining this programatic usage, however their is a cost/benefit tradeoff associated with each. Understanding this paradigm is critical for developing and serving efficient GPU accelerated software at scale.

Reproducibility

See readme.

References

[1] OpenAI, *Triton documentation*, https://triton-lang.org/main/.

[2] PyTorch contributors, *PyTorch documentation*, https://pytorch.org/docs/stable/.

[3] NVIDIA, *CUDA C++ Programming Guide*, https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[4] V. Strassen, *Gaussian elimination is not optimal*, *Numerische Mathematik* 13 (1969), 354–356. https://doi.org/10.1007/BF02165411.

[5] OpenAI, *An Upper Bound of 9/4 for the Matrix Multiplication Exponent*, preprint, October 2, 2026. https://github.com/openai/math/blob/main/preprints/Matrix-Multiplication-Nine-Fourths-October-2-2026/build/paper.tex.